# Software Requirements Specification (SRS)

## JusticeChain - Blockchain-Based Justice Management System

**Version:** 1.0
**Date:** March 11, 2026
**Prepared By:** Janani k (110822104031) **Project:** JusticeChain - Transparent and Secure Justice Management Platform **Guide:** MRS.Rajalakshmi Assistant Professor

## Document Revision History

| Version | Date | Description | Author |
|---|---|---|---|
| 1.0 | March 3, 2026 | Initial SRS Document | Development Team |

## Table of Contents


1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Definitions, Acronyms, and Abbreviations
    - 1.4 References
    - 1.5 Overview
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 Assumptions and Dependencies
3. System Features
4. External Interface Requirements
5. Non-Functional Requirements
6. Other Requirements
7. Appendix


## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the JusticeChain system - a blockchain-based justice management platform. It details the functional and non-

functional requirements for the system, intended for developers, project managers, testers, and stakeholders involved in the development and deployment of the application.

## 1.2 Scope

**JusticeChain** is a decentralized justice management system that leverages blockchain technology to ensure transparency, immutability, and accountability in the judicial process. The system enables:

- **Secure Case Management**: Police officers can register and manage criminal cases
- **Evidence Handling**: Forensic officers can upload and analyze digital evidence with tamper-proof storage
- **Judicial Review**: Judges can review cases, evidence, and provide verdicts
- **Administrative Oversight**: System administrators can manage users, approve cases, and monitor system health
- **Blockchain Integration**: All critical operations are recorded on the Ethereum blockchain for immutability
- **IPFS Storage**: Evidence files are stored on IPFS (InterPlanetary File System) for decentralized storage

**Benefits:**

- Enhanced transparency in the justice system
- Immutable audit trails for all actions
- Secure evidence storage with chain of custody tracking
- Role-based access control for data security
- Automated workflow management
- Real-time case status tracking

**Out of Scope:**

- Physical evidence management
- Court scheduling systems
- Legal document generation
- Financial transaction processing
- Mobile native applications (current scope is web-based)

## 1.3 Definitions, Acronyms, and Abbreviations

| Term | Definition |
|---|---|
| **RBAC** | Role-Based Access Control |
| **IPFS** | InterPlanetary File System - Decentralized storage protocol |
| **Smart Contract** | Self-executing contracts with terms written in code on blockchain |
| **Wallet** | Cryptocurrency wallet address (Ethereum address) |
| **JWT** | JSON Web Token - Authentication token standard |
| **MongoDB** | NoSQL document database |
| **REST API** | Representational State Transfer Application Programming Interface |

| Term | Definition |
|---|---|
| **DApp** | Decentralized Application |
| **Chain of Custody** | Chronological documentation of evidence handling |
| **Web3** | Decentralized web based on blockchain technology |
| **Ethereum** | Blockchain platform for decentralized applications |
| **Gas** | Fee required to execute operations on Ethereum blockchain |
| **Tamper Detection** | Mechanism to identify unauthorized evidence modifications |

### 1.4 References

1. IEEE Std 830-1998 - IEEE Recommended Practice for Software Requirements Specifications
2. Ethereum Smart Contract Documentation - https://ethereum.org/en/developers/docs/smart-contracts/
3. IPFS Documentation - https://docs.ipfs.tech/
4. MongoDB Documentation - https://docs.mongodb.com/
5. React Documentation - https://react.dev/
6. Web3.js Documentation - https://web3js.readthedocs.io/

### 1.5 Overview

This SRS document is organized into seven main sections:

- **Section 2** provides an overall description of the JusticeChain system
- **Section 3** details specific system features and functional requirements
- **Section 4** describes external interface requirements
- **Section 5** outlines non-functional requirements
- **Section 6** covers additional requirements
- **Section 7** provides appendices with supplementary information

---

## 2. Overall Description

### 2.1 Product Perspective

JusticeChain is a standalone web-based system that integrates with:

1. **Ethereum Blockchain Network**: For immutable record-keeping and smart contract execution
2. **IPFS Network**: For decentralized evidence file storage
3. **MongoDB Database**: For relational data storage and quick retrieval
4. **Email Service (EmailJS)**: For user notifications and role ID delivery
5. **WalletConnect**: For Ethereum wallet integration

**System Architecture:**

```
┌──────────────────────────────────────────────────────────────┐
│                    Frontend (React)                          │
```

```
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│  │  Admin   │  │  Police  │  │ Forensic │  │  Judge   │ │
│  │Dashboard │  │Dashboard │  │Dashboard │  │Dashboard│ │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘ │
└──────────────────────────────────────────────────────────┘
                        │
                        │ REST API (HTTPS)
┌───────────────────────▼──────────────────────────────────┐
│              Backend (Node.js/Express)                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│  │ Authentication│  │Case Management│  │Evidence Mgmt│   │
│  └──────────────┘  └──────────────┘  └──────────────┘   │
└──────────────────────────────────────────────────────────┘
      │              │                 │
      │              │                 │
┌─────▼──────┐  ┌────▼───────┐    ┌────▼──────────┐
│  MongoDB   │  │ Ethereum   │    │     IPFS      │
│  Database  │  │Blockchain  │    │   Storage     │
└────────────┘  └────────────┘    └───────────────┘
```

## 2.2 Product Functions

The main functions of JusticeChain include:

### 2.2.1 User Management

- User registration with role selection (Admin, Police, Forensic, Judge)
- Secure authentication using JWT tokens
- Role-based access control (RBAC)
- Wallet connection for blockchain operations
- User profile management
- Session management

### 2.2.2 Case Management

- Case registration by police officers
- Draft case saving and editing
- Case approval workflow (Admin review)
- Case status tracking throughout lifecycle
- Forensic officer assignment to cases
- Judge assignment to cases
- Case transfer requests between police stations
- Case closure and archival
- Case timeline and history tracking

### 2.2.3 Evidence Management

- Evidence upload with IPFS storage
- Multiple evidence types support (Document, Image, Video, Audio, Digital)
- Evidence metadata recording
- Chain of custody tracking

- Tamper detection using hash verification
- Blockchain timestamp recording
- Evidence verification status
- Evidence access logging

#### 2.2.4 Forensic Analysis

- Evidence analysis and reporting
- Investigation notes creation
- Analysis status updates
- Forensic report generation
- Evidence verification and validation
- Analysis completion workflow

#### 2.2.5 Judicial Functions

- Case review by judges
- Hearing scheduling
- Verdict recording
- Case notes and observations
- Final judgment documentation

#### 2.2.6 Administrative Functions

- Case approval/rejection
- User management and suspension
- System health monitoring
- Audit log review
- Blockchain status monitoring
- Database performance tracking
- System statistics and analytics

#### 2.2.7 Blockchain Operations

- Role registration on smart contract
- Evidence hash recording
- Transaction history tracking
- Smart contract interaction
- Wallet integration
- Gas fee estimation

### 2.3 User Classes and Characteristics

#### 2.3.1 Administrator (ADMIN)

- **Characteristics**: High technical proficiency, system-wide oversight privileges
- **Responsibilities**:
    - Approve/reject case registrations

    - Manage user accounts
    - Assign forensic officers and judges to cases
    - Monitor system health and performance
    - Review audit logs
    - Manage blockchain configurations
- **Frequency of Use**: Daily, multiple times
- **Security Level**: Highest (Level 4)

#### 2.3.2 Police Officer (POLICE)

- **Characteristics**: Moderate technical proficiency, field operations focus
- **Responsibilities**:
    - Register new criminal cases
    - Upload initial evidence
    - Submit case details and reports
    - Track case status
    - Request case transfers
    - Add witnesses and suspects
- **Frequency of Use**: Daily, varies by case load
- **Security Level**: High (Level 3)

#### 2.3.3 Forensic Officer (FORENSIC)

- **Characteristics**: High technical proficiency, evidence analysis expertise
- **Responsibilities**:
    - Review assigned cases
    - Analyze evidence materials
    - Upload forensic reports
    - Verify evidence integrity
    - Create investigation notes
    - Complete forensic analysis
- **Frequency of Use**: Daily, per assigned cases
- **Security Level**: High (Level 3)

#### 2.3.4 Judge (JUDGE)

- **Characteristics**: Moderate technical proficiency, judicial decision-making authority
- **Responsibilities**:
    - Review case files and evidence
    - Schedule hearings
    - Record verdicts and judgments
    - Close cases
    - Add judicial notes
- **Frequency of Use**: Regular, per scheduled cases
- **Security Level**: High (Level 3)

### 2.4 Operating Environment

#### 2.4.1 Client-Side Requirements

- **Web Browser**: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- **Internet Connection**: Broadband (minimum 5 Mbps)
- **Screen Resolution**: Minimum 1280x720, Recommended 1920x1080
- **JavaScript**: Enabled
- **Ethereum Wallet**: MetaMask or WalletConnect compatible wallet

#### 2.4.2 Server-Side Requirements

- **Operating System**: Linux (Ubuntu 20.04+), Windows Server 2019+
- **Node.js**: Version 16.x or higher
- **MongoDB**: Version 5.0 or higher
- **Ethereum Node**: Sepolia testnet or Ethereum mainnet access
- **IPFS Node**: Local or Pinata service
- **Memory**: Minimum 4GB RAM, Recommended 8GB
- **Storage**: Minimum 20GB SSD
- **Network**: Static IP, HTTPS support

#### 2.4.3 External Services

- **Blockchain Network**: Ethereum (Sepolia testnet or mainnet)
- **IPFS Provider**: Pinata or local IPFS node
- **Email Service**: EmailJS or SMTP server
- **Hosting**: Render.com, Vercel, or equivalent

### 2.5 Design and Implementation Constraints

1. **Blockchain Constraints**:
    - All transactions require gas fees (Ethereum)
    - Blockchain operations are irreversible
    - Network congestion may delay transactions
    - Smart contract deployment is permanent
2. **Storage Constraints**:
    - IPFS files are immutable once uploaded
    - Large file uploads may take significant time
    - IPFS pinning required for persistence
3. **Security Constraints**:
    - HTTPS required for all communications
    - JWT tokens expire after 24 hours
    - Passwords must meet complexity requirements
    - Wallet private keys never stored on server
4. **Technology Constraints**:

    - Frontend: React 18.2+, Vite 5.0+
    - Backend: Node.js 16+, Express 4.18+
    - Database: MongoDB 5.0+
    - Smart Contracts: Solidity 0.8.19

5. **Regulatory Constraints**:
    - Compliance with data protection regulations
    - Evidence handling must meet legal standards
    - Audit trails must be comprehensive
    - User data privacy requirements

6. **Performance Constraints**:
    - Page load time < 3 seconds
    - API response time < 1 second
    - Evidence upload < 5 minutes for files up to 100MB
    - Support for 1000+ concurrent users

## 2.6 Assumptions and Dependencies

**Assumptions**

1. Users have basic computer literacy
2. Users have access to Ethereum wallets
3. Reliable internet connectivity is available
4. Users understand their role responsibilities
5. Legal framework supports blockchain evidence
6. Organization provides necessary infrastructure

**Dependencies**

1. **Ethereum Network**: System depends on Ethereum network availability
2. **IPFS Network**: Evidence storage requires IPFS network access
3. **MongoDB**: Database service must be operational
4. **EmailJS**: Email notifications depend on EmailJS service
5. **Third-party Libraries**:
    - Web3.js/Wagmi for blockchain interaction
    - Mongoose for database operations
    - Express for API server
    - React for frontend
6. **WalletConnect**: Wallet integration requires WalletConnect service

---

# 3. System Features

## 3.1 User Registration and Authentication

### 3.1.1 Description

The system shall provide secure user registration and authentication mechanisms with role-based access control.

#### 3.1.2 Functional Requirements

**FR-1.1**: The system shall allow users to register with the following information:

- Username (unique, 3-50 characters)
- Email address (valid email format, unique)
- Password (minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 number, 1 special character)
- Role selection (ADMIN, POLICE, FORENSIC, JUDGE)
- Wallet address (optional during registration)

**FR-1.2**: The system shall generate a unique Role ID for each registered user and send it via email.

**FR-1.3**: The system shall hash passwords using bcrypt before storing in database.

**FR-1.4**: The system shall provide login functionality using email/username and password.

**FR-1.5**: The system shall generate JWT tokens upon successful authentication with 24-hour expiration.

**FR-1.6**: The system shall allow users to connect their Ethereum wallet using WalletConnect or MetaMask.

**FR-1.7**: The system shall validate JWT tokens on all protected API endpoints.

**FR-1.8**: The system shall provide logout functionality that clears local session data.

**FR-1.9**: The system shall display password strength indicator during registration.

**FR-1.10**: The system shall prevent duplicate registrations using the same email or username.

#### 3.1.3 Priority

High - Critical for system security and access control

---

### 3.2 Case Management

#### 3.2.1 Description

The system shall provide comprehensive case management functionality for the entire case lifecycle.

#### 3.2.2 Functional Requirements

**FR-2.1**: Police officers shall be able to register new cases with:

- Case title (required)
- Case number (auto-generated or manual)
- Case description (required)
- Incident location
- Incident date and time
- Police station name

**FR-2.2**: The system shall auto-generate unique case IDs in format: `CASE-YYYYMMDD-XXXX`

**FR-2.3**: Police officers shall be able to save cases as drafts before final submission.

**FR-2.4**: The system shall support the following case statuses:

- `DRAFT` - Case being prepared
- `REGISTERED` - Case submitted
- `PENDING_APPROVAL` - Awaiting admin approval
- `APPROVED` - Approved by admin
- `IN_FORENSIC_ANALYSIS` - Under forensic review
- `ANALYSIS_COMPLETE` - Forensic analysis done
- `HEARING` - Case in hearing
- `CLOSED` - Case concluded

**FR-2.5**: Administrators shall be able to approve or reject pending cases.

**FR-2.6**: Administrators shall be able to assign forensic officers to approved cases.

**FR-2.7**: Administrators shall be able to assign judges to cases ready for hearing.

**FR-2.8**: The system shall maintain complete case timeline with all status changes.

**FR-2.9**: Police officers shall be able to request case transfers between stations with:

- Reason for transfer
- Destination station
- Admin approval required

**FR-2.10**: The system shall allow adding suspects to cases with:

- Name, age, gender
- Address
- Physical description
- Criminal history

**FR-2.11**: The system shall allow adding witnesses to cases with:

- Name, contact information
- Statement details
- Credibility assessment

**FR-2.12**: Judges shall be able to schedule hearings with date and time.

**FR-2.13**: The system shall display all cases assigned to logged-in user based on their role.

**FR-2.14**: The system shall provide case search and filter functionality.

**FR-2.15**: The system shall record all case modifications with user and timestamp.

#### 3.2.3 Priority

High - Core functionality for justice management

## 3.3 Evidence Management

### 3.3.1 Description

The system shall provide secure, tamper-proof evidence management with IPFS storage and blockchain verification.

### 3.3.2 Functional Requirements

**FR-3.1**: Users shall be able to upload evidence files up to 100MB with metadata:

- Evidence title (required)
- Evidence description (required)
- Evidence type (DOCUMENT, IMAGE, VIDEO, AUDIO, DIGITAL, PHYSICAL, OTHER)
- Collection date and location
- Associated case ID

**FR-3.2**: The system shall upload evidence files to IPFS and store the IPFS hash.

**FR-3.3**: The system shall generate SHA-256 hash of evidence files for integrity verification.

**FR-3.4**: The system shall record evidence upload transaction on Ethereum blockchain.

**FR-3.5**: The system shall auto-generate unique evidence IDs in format: `EVD-YYYYMMDD-XXXX`

**FR-3.6**: The system shall maintain chain of custody for each evidence with:

- Action performed (UPLOADED, ACCESSED, VERIFIED, LOCKED, TRANSFERRED)
- User who performed action
- User role
- Timestamp
- Action details
- File hash at time of action

**FR-3.7**: The system shall support evidence verification by comparing current hash with original hash.

**FR-3.8**: The system shall trigger tamper detection alert if hash mismatch is detected.

**FR-3.9**: The system shall allow evidence status assignment:

- `PENDING_VERIFICATION`
- `VERIFIED`
- `TAMPERED`
- `LOCKED`

**FR-3.10**: The system shall prevent deletion of verified evidence.

**FR-3.11**: The system shall log all evidence access attempts with user and timestamp.

**FR-3.12**: Forensic officers shall be able to mark evidence as verified after analysis.

**FR-3.13**: The system shall display evidence preview for supported file types.

**FR-3.14**: The system shall provide evidence download functionality for authorized users.

**FR-3.15**: The system shall show blockchain transaction details for evidence uploads.

**FR-3.16**: The system shall display IPFS gateway URL for evidence files.

#### 3.3.3 Priority

Critical - Core security and integrity feature

---

### 3.4 Forensic Analysis

#### 3.4.1 Description

Forensic officers shall be able to analyze cases and evidence, creating detailed reports.

#### 3.4.2 Functional Requirements

**FR-4.1**: Forensic officers shall see only cases assigned to them.

**FR-4.2**: Forensic officers shall be able to view all evidence associated with assigned cases.

**FR-4.3**: Forensic officers shall be able to create investigation notes with:

- Note content
- Timestamp
- Associated case and evidence
- Confidentiality level

**FR-4.4**: Forensic officers shall be able to update evidence analysis status.

**FR-4.5**: Forensic officers shall be able to upload forensic reports as evidence.

**FR-4.6**: Forensic officers shall be able to mark analysis as complete.

**FR-4.7**: The system shall notify admin and judge when forensic analysis is complete.

**FR-4.8**: Forensic officers shall be able to request additional evidence from police.

**FR-4.9**: The system shall update case status to `ANALYSIS_COMPLETE` upon forensic completion.

**FR-4.10**: Forensic officers shall be able to add technical findings and recommendations.

#### 4.4.3 Priority

High - Essential for case processing workflow

---

### 3.5 Judicial Functions

#### 3.5.1 Description

Judges shall be able to review cases, schedule hearings, and record verdicts.

#### 3.5.2 Functional Requirements

**FR-5.1**: Judges shall see only cases assigned to them.

**FR-5.2**: Judges shall be able to view complete case file including:

- Case details
- All evidence
- Forensic reports
- Investigation notes
- Case timeline

**FR-5.3**: Judges shall be able to schedule hearings with:

- Hearing date
- Hearing time
- Court room details
- Attendee list

**FR-5.4**: Judges shall be able to add case notes and observations.

**FR-5.5**: Judges shall be able to record verdicts with:

- Verdict text
- Verdict date
- Judgment reasoning
- Sentencing details (if applicable)

**FR-5.6**: Judges shall be able to close cases after final verdict.

**FR-5.7**: The system shall update case status to `CLOSED` upon judge closure.

**FR-5.8**: Judges shall be able to request additional investigation if needed.

**FR-5.9**: The system shall record all judicial actions on blockchain for transparency.

**FR-5.10**: Judges shall be able to view case hearing history.

#### 3.5.3 Priority

High - Critical for judicial workflow

---

### 3.6 Administrative Functions

#### 3.6.1 Description

Administrators shall have comprehensive system oversight and management capabilities.

### 3.6.2 Functional Requirements

**FR-6.1**: Admin dashboard shall display real-time statistics:

- Total cases (all statuses)
- Pending approval count
- In forensic analysis count
- Closed cases count
- Total users count
- Ready for hearing count
- Total evidence items count

**FR-6.2**: Administrators shall be able to view all cases in the system.

**FR-6.3**: Administrators shall approve or reject pending case registrations with reason.

**FR-6.4**: Administrators shall assign forensic officers to cases by entering forensic role ID.

**FR-6.5**: Administrators shall assign judges to cases by entering judge role ID.

**FR-6.6**: Administrators shall be able to view all registered users with:

- Username
- Email
- Role
- Role ID
- Wallet address
- Registration date
- Account status

**FR-6.7**: Administrators shall be able to suspend/unsuspend user accounts.

**FR-6.8**: Administrators shall be able to view audit logs showing:

- Timestamp
- User
- Action performed
- Action details
- Success/failure status

**FR-6.9**: Administrators shall monitor blockchain status:

- Smart contract address
- Contract status
- Network status (Connected/Disconnected)
- Chain ID

**FR-6.10**: Administrators shall monitor system health:

- Database status
- IPFS connectivity
- Blockchain node status

- Server uptime
- API response time
- Database performance

**FR-6.11**: Administrators shall view case status distribution analytics.

**FR-6.12**: The system shall auto-refresh admin dashboard every 30 seconds.

**FR-6.13**: Administrators shall be able to view evidence verification statistics.

**FR-6.14**: Administrators shall receive notifications for critical system events.

**FR-6.15**: Administrators shall be able to manually trigger data refresh.

#### 3.6.3 Priority

High - Essential for system administration

---

### 3.7 Blockchain Integration

#### 3.7.1 Description

The system shall integrate with Ethereum blockchain for immutable record-keeping.

#### 3.7.2 Functional Requirements

**FR-7.1**: The system shall deploy JusticeChain smart contract with functions:

- `registerPolice(address)` - Register police officer wallet
- `registerForensic(address)` - Register forensic officer wallet
- `registerJudge(address)` - Register judge wallet
- `hasAnyRole(address)` - Check if wallet has assigned role

**FR-7.2**: The system shall automatically register user wallet on blockchain upon wallet connection.

**FR-7.3**: The system shall record evidence uploads on blockchain with:

- Evidence ID
- IPFS hash
- Uploader wallet address
- Timestamp

**FR-7.4**: The system shall record case status changes on blockchain.

**FR-7.5**: The system shall display transaction hash for all blockchain operations.

**FR-7.6**: The system shall estimate gas fees before transaction submission.

**FR-7.7**: The system shall handle transaction failures gracefully with error messages.

**FR-7.8**: The system shall support both Sepolia testnet and Ethereum mainnet.

**FR-7.9**: The system shall maintain transaction history for auditing.

**FR-7.10**: The system shall verify smart contract status on dashboard.

**FR-7.11**: Users shall be able to view their blockchain transaction history.

**FR-7.12**: The system shall prevent operations if wallet is not connected.

#### 3.7.3 Priority

Critical - Core blockchain functionality

---

### 3.8 Notifications

#### 3.8.1 Description

The system shall provide email notifications for important events.

#### 3.8.2 Functional Requirements

**FR-8.1**: The system shall send email notification upon successful registration containing:

- Welcome message
- Unique Role ID
- Login instructions
- System URL

**FR-8.2**: The system shall send notifications when:

- Case is approved/rejected
- Forensic officer is assigned to case
- Judge is assigned to case
- Hearing is scheduled
- verdict is recorded
- Case is closed

**FR-8.3**: Email notifications shall include relevant case details and action items.

**FR-8.4**: The system shall use EmailJS or NodeMailer for email delivery.

**FR-8.5**: Failed email delivery shall be logged but not prevent operation completion.

#### 3.8.3 Priority

Medium - Enhances user experience but not critical

---

### 3.9 Reporting

#### 3.9.1 Description

The system shall generate various reports for analysis and auditing.

#### 3.9.2 Functional Requirements

**FR-9.1**: The system shall generate case summary reports including:

- Case details
- Timeline
- Involved parties
- Evidence list
- Current status

**FR-9.2**: Forensic officers shall generate investigation reports with findings.

**FR-9.3**: Administrators shall generate system usage reports showing:

- User activity statistics
- Case processing times
- Evidence upload volumes
- System performance metrics

**FR-9.4**: The system shall generate audit trail reports for any date range.

**FR-9.5**: Reports shall be exportable in PDF or CSV format.

#### 3.9.3 Priority

Low - Useful but not immediately critical

---

# 4. External Interface Requirements

## 4.1 User Interface Requirements

#### 4.1.1 General UI Requirements

**UI-1**: The system shall provide a responsive web interface compatible with desktop and tablet devices.

**UI-2**: The system shall use consistent navigation across all pages with sticky header.

**UI-3**: The system shall apply Tailwind CSS for styling with consistent color scheme:

- Primary: Blue (#3B82F6)
- Success: Green (#10B981)
- Warning: Yellow (#F59E0B)
- Error: Red (#EF4444)
- Neutral: Gray shades

**UI-4**: The system shall display loading indicators during asynchronous operations.

**UI-5**: The system shall show toast notifications for user actions (success, error, info).

**UI-6**: All forms shall include client-side validation with error messages.

**UI-7**: The system shall display user's role and username in navigation bar.

**UI-8**: The system shall provide logout button visible on all authenticated pages.

#### 4.1.2 Page-Specific Requirements

**UI-9**: **Home Page** shall display:

- Hero section with system overview
- Feature highlights
- Call-to-action buttons
- Statistics overview

**UI-10**: **Registration Page** shall display:

- Registration form with all required fields
- Role selection dropdown
- Password strength indicator
- Terms and conditions checkbox

**UI-11**: **Login Page** shall display:

- Login form (email/username, password)
- "Remember me" option
- "Forgot password" link
- Link to registration page

**UI-12**: **Dashboard Pages** shall display:

- Role-specific navigation tabs
- Statistics cards
- Data tables with sorting and filtering
- Action buttons based on permissions

**UI-13**: **Case Detail View** shall display:

- Case information panel
- Evidence list with thumbnails
- Timeline visualization
- Action buttons based on user role

**UI-14**: **Evidence Upload Modal** shall display:

- File upload drag-and-drop area
- File type selector
- Metadata input fields
- Upload progress bar
- IPFS hash upon completion

### 4.2 Hardware Interface Requirements

**HW-1**: The system shall support standard keyboard and mouse input devices.

**HW-2**: The system shall support touch input for tablet devices.

**HW-3**: The system shall utilize client GPU for blockchain transaction signing.

## 4.3 Software Interface Requirements

### 4.3.1 Database Interface

**SW-1**: The system shall use MongoDB 5.0+ for data persistence using Mongoose ODM.

**SW-2**: The system shall implement the following database models:

- User (username, email, password, role, roleId, wallet, isVerified, isSuspended)
- Case (caseId, title, description, status, registeredBy, assignedForensic, assignedJudge, timeline)
- Evidence (evidenceId, caseId, type, title, uploadedBy, ipfsHash, fileHash, chainOfCustody)
- InvestigationNote (caseId, forensicOfficer, content, timestamp)
- Witness (caseId, name, contactDetails, statement)
- Suspect (caseId, name, age, gender, address, description)
- ActivityLog (user, action, timestamp, details, status)
- UserSession (userId, token, expiresAt)

**SW-3**: Database connection strings shall use environment variables for security.

### 4.3.2 Blockchain Interface

**SW-4**: The system shall interact with Ethereum blockchain via Web3.js or Wagmi library.

**SW-5**: The system shall connect to Ethereum networks:

- Sepolia Testnet (Chain ID: 11155111) for testing
- Ethereum Mainnet (Chain ID: 1) for production

**SW-6**: The system shall use smart contract ABI (Application Binary Interface) stored in `utils/JusticeChainABI.json`.

**SW-7**: The system shall use Infura or Alchemy as Ethereum node provider.

### 4.3.3 IPFS Interface

**SW-8**: The system shall integrate with IPFS using Pinata API or local IPFS node.

**SW-9**: The system shall use Pinata JWT token for authentication (Pinata service).

**SW-10**: Evidence files shall be uploaded to IPFS and pinned for persistence.

**SW-11**: The system shall generate IPFS gateway URLs in format: `https://gateway.pinata.cloud/ipfs/{hash}`

### 4.3.4 Email Service Interface

**SW-12**: The system shall use EmailJS API for sending emails.

**SW-13**: Email service credentials shall be stored in environment variables:

- EMAILJS_SERVICE_ID
- EMAILJS_TEMPLATE_ID
- EMAILJS_PUBLIC_KEY
- EMAILJS_PRIVATE_KEY

**SW-14**: The system shall handle email service failures without blocking primary operations.

#### 4.3.5 Wallet Interface

**SW-15**: The system shall integrate with Ethereum wallets via WalletConnect protocol.

**SW-16**: The system shall support MetaMask browser extension.

**SW-17**: The system shall request wallet connection using Web3Modal or similar library.

**SW-18**: The system shall never request or store private keys.

### 4.4 Communication Interface Requirements

**COMM-1**: All client-server communication shall use HTTPS protocol in production.

**COMM-2**: REST API endpoints shall accept and return JSON format data.

**COMM-3**: API requests shall include JWT token in Authorization header: `Bearer {token}`

**COMM-4**: WebSocket connections may be used for real-time notifications (future enhancement).

**COMM-5**: CORS (Cross-Origin Resource Sharing) shall be configured to allow frontend domain.

**COMM-6**: API rate limiting shall be implemented to prevent abuse (100 requests/minute per user).

---

## 5. Non-Functional Requirements

### 5.1 Performance Requirements

**PERF-1**: Page load time shall not exceed 3 seconds on standard broadband connection.

**PERF-2**: API response time for queries shall not exceed 1 second for 95% of requests.

**PERF-3**: Evidence upload shall complete within 5 minutes for files up to 100MB.

**PERF-4**: The system shall support minimum 100 concurrent users without performance degradation.

**PERF-5**: Database queries shall be optimized with appropriate indexing (caseId, evidenceId, email, username).

**PERF-6**: Dashboard statistics shall refresh within 2 seconds.

**PERF-7**: Blockchain transaction submission shall complete within 30 seconds (excluding network confirmation time).

**PERF-8**: The system shall cache frequently accessed data (case lists, user profiles) for 5 minutes.

## 5.2 Security Requirements

**SEC-1**: All passwords shall be hashed using bcrypt with minimum 10 salt rounds.

**SEC-2**: JWT tokens shall expire after 24 hours requiring re-authentication.

**SEC-3**: The system shall implement role-based access control (RBAC) on all protected routes.

**SEC-4**: The system shall validate all user inputs to prevent SQL injection and XSS attacks.

**SEC-5**: File uploads shall be scanned for malware before IPFS upload.

**SEC-6**: The system shall enforce HTTPS for all production communications.

**SEC-7**: Sensitive data (passwords, tokens) shall never be logged.

**SEC-8**: The system shall implement CORS restrictions to prevent unauthorized domain access.

**SEC-9**: API endpoints shall validate authentication tokens on every request.

**SEC-10**: Private keys shall never be transmitted or stored on backend servers.

**SEC-11**: The system shall implement rate limiting to prevent brute force attacks.

**SEC-12**: Environment variables shall be used for all sensitive configuration data.

**SEC-13**: Database connections shall use strong passwords and restricted network access.

**SEC-14**: Failed login attempts shall be logged and may trigger account lockout after 5 attempts.

**SEC-15**: All blockchain transactions shall require user confirmation via wallet.

## 5.3 Reliability Requirements

**REL-1**: The system shall have 99.5% uptime availability (excluding planned maintenance).

**REL-2**: Planned maintenance windows shall be announced 48 hours in advance.

**REL-3**: The system shall implement automatic database backups every 24 hours.

**REL-4**: Critical errors shall trigger admin notifications via email.

**REL-5**: The system shall gracefully handle blockchain network failures without data loss.

**REL-6**: Failed blockchain transactions shall be queued for retry up to 3 attempts.

**REL-7**: Database connection failures shall trigger automatic reconnection attempts.

**REL-8**: The system shall implement error logging to track all exceptions.

## 5.4 Availability Requirements

**AVAIL-1**: The system shall be accessible 24/7 except during scheduled maintenance.

**AVAIL-2**: Maximum planned downtime shall not exceed 4 hours per month.

**AVAIL-3**: Critical security patches shall be applied within 48 hours of release.

**AVAIL-4**: The system shall implement health check endpoints for monitoring.

### 5.5 Maintainability Requirements

**MAINT-1**: Code shall follow consistent style guide (Eslint configuration).

**MAINT-2**: All functions shall include JSDoc comments describing purpose and parameters.

**MAINT-3**: Complex logic shall include inline comments explaining approach.

**MAINT-4**: Database schema changes shall use migration scripts.

**MAINT-5**: The system shall maintain comprehensive error logs for debugging.

**MAINT-6**: Version control (Git) shall be used for all code changes.

**MAINT-7**: The system shall separate configuration from code using environment variables.

### 5.6 Portability Requirements

**PORT-1**: The frontend shall be compatible with Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.

**PORT-2**: The backend shall run on Linux, Windows, and macOS operating systems.

**PORT-3**: The system shall support deployment on cloud platforms (Render, Vercel, AWS, Azure).

**PORT-4**: Database connection strings shall be configurable without code changes.

**PORT-5**: The system shall support both testnet and mainnet blockchain networks via configuration.

### 5.7 Usability Requirements

**USE-1**: New users shall be able to register and submit first case within 15 minutes.

**USE-2**: Navigation shall be intuitive with maximum 3 clicks to reach any function.

**USE-3**: Error messages shall be clear and actionable, guiding users to resolution.

**USE-4**: The system shall provide helpful tooltips for complex operations.

**USE-5**: Form validation feedback shall appear in real-time as user types.

**USE-6**: The system shall maintain consistent terminology throughout the interface.

**USE-7**: Color coding shall be used to indicate status (green=success, red=error, yellow=warning).

**USE-8**: Loading states shall indicate progress for long-running operations.

### 5.8 Scalability Requirements

**SCALE-1**: The system architecture shall support horizontal scaling of backend servers.

**SCALE-2**: Database shall support sharding for datasets exceeding 100GB.

**SCALE-3**: The system shall handle 1000+ concurrent users with load balancing.

**SCALE-4**: Static assets shall be served via CDN for improved global performance.

**SCALE-5**: IPFS file storage shall scale to accommodate 10TB+ of evidence data.

### 5.9 Compliance Requirements

**COMP-1**: The system shall comply with GDPR data protection regulations.

**COMP-2**: User data deletion requests shall be processed within 30 days.

**COMP-3**: The system shall maintain audit trails for minimum 7 years.

**COMP-4**: Evidence handling shall meet legal chain of custody requirements.

**COMP-5**: The system shall provide data export functionality for compliance requests.

---

## 6. Other Requirements

### 6.1 Legal Requirements

**LEGAL-1**: The system shall display terms of service requiring user acceptance during registration.

**LEGAL-2**: Privacy policy shall be accessible from all pages.

**LEGAL-3**: The system shall obtain user consent for data processing (GDPR compliance).

**LEGAL-4**: Evidence admissibility shall meet jurisdictional legal standards.

### 6.2 Internationalization Requirements

**I18N-1**: The system shall support English language in initial release.

**I18N-2**: Date and time formats shall be configurable based on locale.

**I18N-3**: The system architecture shall support future multi-language expansion.

### 6.3 Documentation Requirements

**DOC-1**: User manual shall be provided covering all system features.

**DOC-2**: API documentation shall be maintained using tools like Swagger/OpenAPI.

**DOC-3**: Smart contract functions shall be documented with NatSpec comments.

**DOC-4**: Setup and deployment guides shall be provided for administrators.

**DOC-5**: Code repositories shall include comprehensive README files.

### 6.4 Training Requirements

**TRAIN-1**: Administrator training materials shall be provided.

**TRAIN-2**: Video tutorials shall be created for common user workflows.

**TRAIN-3**: FAQ section shall address common user questions.

---

# 7. Appendix

## 7.1 Glossary

| Term | Definition |
|---|---|
| **Blockchain** | Distributed ledger technology ensuring immutable record-keeping |
| **Chain of Custody** | Documentation tracking evidence handling from collection to presentation |
| **DApp** | Decentralized application running on blockchain |
| **Gas Fee** | Transaction cost on Ethereum network |
| **Hash** | Fixed-size cryptographic output from input data |
| **IPFS** | Peer-to-peer distributed file system |
| **JWT** | JSON Web Token for stateless authentication |
| **MetaMask** | Browser extension for Ethereum wallet management |
| **Smart Contract** | Self-executing code on blockchain |
| **Tamper Detection** | Process to identify unauthorized modifications |
| **Web3** | Decentralized internet leveraging blockchain |

## 7.2 Analysis Models

### 7.2.1 Use Case Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                    JusticeChain System                       │
│                                                              │
│  ┌──────────────┐                                            │
│  │   Register   │◄──────────────────┐                        │
│  │   & Login    │                   │                        │
│  └──────────────┘                   │                        │
│                                     │                        │
│  ┌──────────────┐          ┌────────┴──────────┐             │
│  │   Manage     │◄─────────┤       ADMIN       │             │
│  │   Users      │          └───────────────────┘             │
│  └──────────────┘                                            │
│                                                              │
│  ┌──────────────┐          ┌───────────────────┐             │
│  │   Submit     │◄─────────┤       POLICE      │             │
│  │   Cases      │          └───────────────────┘             │
```

```
│   └──────────────┘                                          │
│                                                              │
│   ┌──────────────┐       ┌──────────────────────┐           │
│   │   Analyze    │◄──────┤       FORENSIC       │           │
│   │  Evidence    │       └──────────────────────┘           │
│   └──────────────┘                                          │
│                                                              │
│   ┌──────────────┐       ┌──────────────────────┐           │
│   │   Record     │◄──────┤        JUDGE         │           │
│   │  Verdict     │       └──────────────────────┘           │
│   └──────────────┘                                          │
│                                                              │
│   ┌──────────────┐                                          │
│   │  Blockchain  │                                          │
│   │ Integration  │◄──────────────────────────────All Users  │
│   └──────────────┘                                          │
└──────────────────────────────────────────────────────────────┘
```

### 7.2.2 Data Flow Diagram (Level 0)

```
External Entities:
- Users (Police, Forensic, Judge, Admin)
- Ethereum Blockchain
- IPFS Network
- Email Service

                        ┌──────────────────┐
                        │      Users       │
                        └──────────────────┘
                                 │
                                 ▼
┌──────────────┐        ┌──────────────────┐        ┌──────────────┐
│  Ethereum    │◄───────┤  JusticeChain    │────────►│   MongoDB    │
│ Blockchain   │        │     System       │        │  Database    │
└──────────────┘        └──────────────────┘        └──────────────┘
                                 ▲
                                 │
                        ┌──────────────────┐
                        │   IPFS Network   │
                        │  Email Service   │
                        └──────────────────┘
```

### 7.2.3 Entity-Relationship Diagram

```
┌──────────┐          ┌──────────┐          ┌──────────┐
│   USER   │          │   CASE   │          │ EVIDENCE │
├──────────┤          ├──────────┤          ├──────────┤
│ userId   │──────────►│ caseId   │──────────►│evidenceId│
```

```
│ username │ creates │ title     │ contains│ type      │
│ email    │         │ status    │         │ ipfsHash  │
│ role     │◄────────│assignedTo │◄────────│uploadedBy │
│ roleId   │ assigned│registBy   │ uploads │ fileHash  │
└──────────┘         └───────────┘         └───────────┘
     │                   │                     │
     │                   │                     │
     ▼                   ▼                     ▼
┌──────────┐        ┌──────────┐        ┌──────────┐
│ SESSION  │        │ TIMELINE │        │CHAIN_OF_ │
├──────────┤        ├──────────┤        │ CUSTODY  │
│ token    │        │ status   │        ├──────────┤
│ expiresAt│        │ timestamp│        │ action   │
└──────────┘        │ performBy│        │ timestamp│
                    └──────────┘        │ performer│
                                        └──────────┘
```

## 7.3 Technology Stack Summary

**Frontend**

- **Framework**: React 18.2.0
- **Build Tool**: Vite 5.0.0
- **Routing**: React Router DOM 6.20.0
- **Styling**: Tailwind CSS 3.3.0
- **Blockchain**: Wagmi 2.12.17, Viem 2.21.19
- **Wallet**: Web3Modal 5.1.11
- **Notifications**: React Toastify 11.0.5
- **HTTP Client**: Fetch API

**Backend**

- **Runtime**: Node.js 16+
- **Framework**: Express 4.18.2
- **Database ODM**: Mongoose 7.7.0
- **Authentication**: JSON Web Token 9.0.2
- **Password Hashing**: Bcryptjs 2.4.3
- **File Upload**: Multer 2.0.2
- **Email**: EmailJS 5.0.2, Nodemailer 6.9.7
- **Blockchain**: Web3.js 4.16.0
- **CORS**: CORS 2.8.5

**Database**

- **Primary DB**: MongoDB 5.0+
- **Hosting**: MongoDB Atlas or self-hosted

**Blockchain**

- **Network**: Ethereum (Sepolia Testnet / Mainnet)
- **Smart Contract**: Solidity 0.8.19
- **Libraries**: Ethers.js 6.9.0, Web3.js 4.16.0

**Storage**

- **Decentralized**: IPFS via Pinata API

**DevOps**

- **Version Control**: Git
- **CI/CD**: GitHub Actions (potential)
- **Hosting**: Render.com (backend), Vercel (frontend)
- **Monitoring**: Custom health checks

## 7.4 API Endpoint Summary

**Authentication Routes (`/api/auth`)**

- `POST /register` - User registration
- `POST /login` - User authentication
- `POST /logout` - Session termination
- `POST /connect-wallet` - Link Ethereum wallet

**Case Routes (`/api/cases`)**

- `GET /all` - List all cases
- `GET /my-cases` - User's assigned cases
- `GET /:id` - Case details
- `POST /` - Create new case
- `PUT /:id` - Update case
- `PUT /:id/approve` - Approve case (Admin)
- `POST /assign-forensic` - Assign forensic officer
- `POST /assign-judge` - Assign judge
- `PUT /:id/close` - Close case
- `POST /transfer-request` - Request case transfer

**Evidence Routes (`/api/evidence`)**

- `GET /case/:caseId` - List evidence for case
- `GET /:id` - Evidence details
- `POST /upload` - Upload evidence to IPFS
- `PUT /:id/verify` - Mark evidence as verified
- `GET /:id/chain-of-custody` - View custody history

**User Routes (`/api/users`)**

- `GET /` - List all users (Admin)

- `GET /:id` - User details
- `PUT /:id/suspend` - Suspend user account
- `PUT /:id/role` - Update user role

**Admin Routes (`/api/admin`)**

- `GET /stats/cases` - Case statistics
- `GET /stats/users` - User statistics
- `GET /audit-logs` - System audit logs
- `GET /health` - System health status
- `GET /blockchain-status` - Blockchain connection status

**Investigation Routes (`/api/investigations`)**

- `POST /notes` - Create investigation note
- `GET /case/:caseId/notes` - List case notes
- `PUT /analysis-complete` - Mark analysis complete

**Report Routes (`/api/reports`)**

- `POST /` - Create report
- `GET /case/:caseId` - Case reports
- `GET /:id/download` - Download report

### 7.5 Environment Variables

```
# Backend Environment Variables
PORT=5000
MONGODB_URI=mongodb://localhost:27017/justicechain
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRE=24h

# Blockchain Configuration
ETHEREUM_NETWORK=sepolia
INFURA_PROJECT_ID=your_infura_project_id
CONTRACT_ADDRESS=0x...
BACKEND_PRIVATE_KEY=0x...

# IPFS/Pinata Configuration
PINATA_API_KEY=your_pinata_api_key
PINATA_SECRET_API_KEY=your_pinata_secret_key
PINATA_JWT=your_pinata_jwt_token

# Email Configuration
EMAILJS_SERVICE_ID=your_service_id
EMAILJS_TEMPLATE_ID=your_template_id
EMAILJS_PUBLIC_KEY=your_public_key
EMAILJS_PRIVATE_KEY=your_private_key

# Frontend Environment Variables
```

```
VITE_API_URL=http://localhost:5000/api
VITE_CONTRACT_ADDRESS=0x...
VITE_CHAIN_ID=11155111
VITE_WALLETCONNECT_PROJECT_ID=your_project_id
```

### 7.6 Database Collections Structure

**Users Collection**

```
{
  _id: ObjectId,
  username: String,
  email: String,
  password: String (hashed),
  role: String (ADMIN|POLICE|FORENSIC|JUDGE),
  roleId: String (unique),
  wallet: String (Ethereum address),
  isVerified: Boolean,
  isSuspended: Boolean,
  createdAt: Date
}
```

**Cases Collection**

```
{
  _id: ObjectId,
  caseId: String (unique),
  title: String,
  description: String,
  caseNumber: String,
  status: String,
  isDraft: Boolean,
  registeredBy: ObjectId (ref: User),
  policeStation: String,
  assignedForensic: ObjectId (ref: User),
  assignedJudge: ObjectId (ref: User),
  approvedBy: ObjectId (ref: User),
  approvedAt: Date,
  timeline: [{
    status: String,
    timestamp: Date,
    performedBy: ObjectId,
    notes: String
  }],
  createdAt: Date,
  updatedAt: Date
}
```

**Evidence Collection**

```
{
  _id: ObjectId,
  evidenceId: String (unique),
  caseId: ObjectId (ref: Case),
  type: String (DOCUMENT|IMAGE|VIDEO|AUDIO|DIGITAL|PHYSICAL|OTHER),
  title: String,
  description: String,
  uploadedBy: ObjectId (ref: User),
  fileName: String,
  fileSize: Number,
  mimeType: String,
  ipfsHash: String,
  fileHash: String (SHA-256),
  blockchainTxHash: String,
  status: String (PENDING_VERIFICATION|VERIFIED|TAMPERED|LOCKED),
  chainOfCustody: [{
    action: String,
    performedBy: ObjectId,
    performedByRole: String,
    timestamp: Date,
    details: String,
    hash: String
  }],
  createdAt: Date
}
```

## 7.7 Smart Contract Functions

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract JusticeChain {
    // Admin management
    address public admin;
    address public backend;

    // Role mappings
    mapping(address => bool) public police;
    mapping(address => bool) public forensic;
    mapping(address => bool) public judge;

    // Events
    event RoleRegistered(address indexed wallet, string role);
    event BackendAddressUpdated(address newBackend);

    // Functions
    function registerPolice(address _addr) public;
    function registerForensic(address _addr) public;
```

```
    function registerJudge(address _addr) public;
    function hasAnyRole(address _addr) public view returns (bool);
    function setBackendAddress(address _backend) public;
}
```

### 7.8 Deployment Architecture

```
┌──────────────────────────────────────────────────────────┐
│                      Production Setup                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Frontend (Vercel)                                       │
│  ├─ React App (Static Build)                             │
│  ├─ CDN Distribution                                     │
│  └─ HTTPS/SSL Enabled                                    │
│                                                          │
│  Backend (Render.com)                                    │
│  ├─ Node.js Server                                       │
│  ├─ Express APIs                                         │
│  ├─ Auto-deploy from Git                                 │
│  └─ Environment Variables Configured                     │
│                                                          │
│  Database (MongoDB Atlas)                                │
│  ├─ Cloud-hosted MongoDB                                 │
│  ├─ Automatic Backups                                    │
│  ├─ Replica Sets                                         │
│  └─ Connection Pooling                                   │
│                                                          │
│  Blockchain (Ethereum)                                   │
│  ├─ Sepolia Testnet (Development)                        │
│  ├─ Ethereum Mainnet (Production)                        │
│  └─ Infura Node Provider                                 │
│                                                          │
│  Storage (Pinata/IPFS)                                   │
│  ├─ IPFS File Storage                                    │
│  ├─ Automatic Pinning                                    │
│  └─ Gateway Access                                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

### 7.9 Security Best Practices

1. **Authentication**
    - Use strong JWT secret keys (minimum 32 characters)
    - Implement token refresh mechanism
    - Set appropriate token expiration times
2. **Password Security**

    - Enforce strong password policies
    - Use bcrypt with minimum 10 rounds
    - Never log or transmit plain passwords

3. **API Security**
    - Validate all inputs
    - Implement rate limiting
    - Use HTTPS in production
    - Configure CORS appropriately

4. **Blockchain Security**
    - Never expose private keys
    - Verify contract addresses
    - Validate transaction data
    - Handle gas estimation carefully

5. **Data Protection**
    - Encrypt sensitive data at rest
    - Use parameterized queries
    - Implement proper access controls
    - Regular security audits

### 7.10 Future Enhancements

1. **Mobile Applications**: Native iOS and Android apps
2. **Multi-language Support**: Internationalization (i18n)
3. **Advanced Analytics**: Machine learning for case insights
4. **Real-time Notifications**: WebSocket implementation
5. **Document OCR**: Automatic text extraction from documents
6. **Video Evidence Analysis**: Automated frame analysis
7. **Biometric Authentication**: Fingerprint/facial recognition
8. **Advanced Reporting**: Customizable report templates
9. **Integration APIs**: Third-party system integrations
10. **AI-Powered Search**: Natural language case search

---

## Approval Signatures

| Role | Name | Signature | Date |
|---|---|---|---|
| **Product Owner** | _____ | _____ | _____ |
| **Project Manager** | _____ | _____ | _____ |
| **Lead Developer** | _____ | _____ | _____ |
| **QA Lead** | _____ | _____ | _____ |
| **Stakeholder** | _____ | _____ | _____ |

---

**Document Status:** APPROVED
**Next Review Date:** June 3, 2026
**Document Custodian:** Development Team
**Classification:** Internal Use

---

*End of Software Requirements Specification Document*